

Lógica Computacional 2024/2025 - TP1

Grupo 6

- Cláudia Faria, a105531
- Patrícia Bastos, a102502

```
In [ ]: !pip install pysmt
!pysmt-install --z3
!pip install ortools
!pip install networkx
```

Exercício 1

Enunciado

Pretende-se construir um horário semanal para o plano de reuniões de projeto de uma "StartUp" de acordo com as seguintes condições:

1. Cada reunião ocupa uma sala (enumeradas $1..S$) durante um "slot" (tempo, dia). Assume-se os dias enumerados $1..D$ e, em cada dia, os tempos enumerados $1..T$.
2. Cada reunião tem associado um projeto (enumerados $1..P$) e um conjunto de participantes. Os diferentes colaboradores são enumerados $1..C$.
3. Cada projeto tem associado um conjunto de colaboradores, dos quais um é o líder. Cada projeto realiza um dado número de reuniões semanais. São "inputs" do problema o conjunto de colaboradores de cada projeto, o seu líder e o número de reuniões semanais.
4. O líder do projeto participa em todas as reuniões do seu projeto; os restantes colaboradores podem ou não participar consoante a sua disponibilidade, num mínimo ("quorum") de 50% do total de colaboradores do projeto. A disponibilidade de cada participante, incluindo o líder, é um conjunto de "slots" ("inputs" do problema).

Variáveis

Inputs

- S - Número total de salas
- D - Número de dias de trabalho
- T - Tempos disponíveis por dia
- P - Número total de projetos
- C - Número total de colaboradores
- *projetos* - constituído pelos projetos p aos quais está associado o respetivo líder, conjunto de colaboradores, e número de reuniões semanais
- *colaboradores* - constituído pelos colaboradores aos quais estão associados os respetivos "slots" (tempo, dia) disponíveis

Auxiliares

- $reuniao_{s,d,t,p}$ - indica se uma reunião do projeto p está agendada na sala s , no dia d , no tempo t
- $colaborador_{c,d,t,p}$ - indica se o colaborador c está a participar na reunião do projeto p , no dia d , no tempo t
- $haReuniao_{d,p}$ - indica se há uma reunião do projeto p no dia d

Em que $d \in [1..D]$, $s \in [1..S]$, $t \in [1..T]$, $p \in [1..P]$, $c \in [1..C]$

Restrições

- Cada reunião ocupa uma sala durante um slot (tempo,dia)
- Cada reunião tem associado um projeto
- Cada reunião tem associado um conjunto de participantes
- Cada projeto realiza um dado número de reuniões semanais
- O líder do projeto participa em todas as reuniões do projeto
- Mínimo de 50% do total de colaboradores do projeto têm de participar da reunião
- Os colaboradores participam na reunião conforme a sua disponibilidade
- Cada colaborador só pode estar numa reunião em cada slot

Definição de inputs

Exemplo 1:

```
In [655]: S = 2
D = 5
T = 5
P = 3
C = 8

projetos = {
    1: (1, [1,3,5,8], 8),
    2: (2, [1,2,4,7], 11),
    3: (3, [2,3,5,6,7,8], 10)
}

colaboradores = {
    1: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 1), (2, 2), (2, 3), (2, 4),
        (2, 5), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (4, 1), (4, 2), (4, 3),
        (4, 4), (4, 5), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)],
    2: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 1), (2, 2), (2, 3), (2, 4),
        (2, 5), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (4, 1), (4, 2), (4, 3),
        (4, 4), (4, 5), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)],
    3: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 1), (2, 2), (2, 3), (2, 4),
```

```

        (2, 5), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (4, 1), (4, 2), (4, 3),
        (4, 4), (4, 5), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)],
4: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 1), (2, 2), (2, 3), (2, 4),
    (2, 5), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (4, 1), (4, 2), (4, 3),
    (4, 4), (4, 5), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)],
5: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 1), (2, 2), (2, 3), (2, 4),
    (2, 5), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (4, 1), (4, 2), (4, 3),
    (4, 4), (4, 5), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)],
6: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 1), (2, 2), (2, 3), (2, 4),
    (2, 5), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (4, 1), (4, 2), (4, 3),
    (4, 4), (4, 5), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)],
7: [(1,2), (2,2), (3,2), (4,2), (5,2)],
8: [(1,1), (2,1), (3,1), (4,1), (5,1)]
}

```

Exemplo 2:

In [638...

```

S = 3
D = 5
T = 4
P = 4
C = 7

projetos = {
    1: (1, [1,3,5], 20),
    2: (2, [1,2,4,7], 11),
    3: (3, [2,3,5,6,7], 10),
    4: (4, [1,2,3,4,5,6,7], 9)
}
colaboradores = {
    1: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 7), (2, 1), (2, 2), (2, 3), (2, 4),
        (2, 5), (2, 6), (2, 7), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6), (4, 1),
        (4, 2), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (5, 1), (5, 2), (5, 3), (5, 4),
        (5, 5), (5, 6), (5, 7)],
    2: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 1), (2, 2), (2, 3),
        (2, 4), (2, 5), (2, 6), (2, 7), (3, 1), (3, 2), (3, 3), (3, 5), (3, 6), (3, 7),
        (4, 1), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (5, 1), (5, 2), (5, 3), (5, 4),
        (5, 5), (5, 6), (5, 7)],
    3: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 1), (2, 3), (2, 4),
        (2, 5), (2, 6), (2, 7), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6), (3, 7),
        (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (5, 1), (5, 2), (5, 3),
        (5, 4), (5, 5), (5, 6)],
    4: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 1), (2, 2), (2, 3),
        (2, 4), (2, 5), (2, 6), (2, 7), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6),
        (3, 7), (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (5, 1), (5, 2),
        (5, 3), (5, 4), (5, 5), (5, 6), (5, 7)],
    5: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (2, 1), (2, 2), (2, 3),
        (2, 4), (2, 5), (2, 6), (2, 7), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6),
        (3, 7), (4, 1), (4, 2), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (5, 1), (5, 2),
        (5, 3), (5, 4), (5, 5), (5, 6), (5, 7)],
    6: [(1, 1), (1, 2), (1, 3), (1, 5), (1, 6), (1, 7), (2, 1), (2, 2), (2, 3), (2, 4),
        (2, 5), (2, 6), (2, 7), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6), (3, 7), (4, 2),
        (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5),
        (5, 6), (5, 7)],
    7: [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (1, 7), (2, 2), (2, 3), (2, 4), (2, 5),
        (2, 6), (2, 7), (3, 1), (3, 2), (3, 3), (3, 4), (3, 5), (3, 6), (3, 7), (4, 1),
        (4, 2), (4, 3), (4, 4), (4, 5), (4, 6), (4, 7), (5, 1), (5, 2), (5, 3), (5, 4),
        (5, 5), (5, 6), (5, 7)]
}

```

Exemplo 3:

Não é possível devido à falta de horários dos colaboradores disponíveis.

In [543...

```

S = 2
D = 2
T = 2
P = 3
C = 3

projetos = {
    1: (1, [1,3,2], 2),
    2: (2, [1,2], 2),
    3: (3, [1,3,2], 2)
}

colaboradores = {
    1: [(1,2), (2,1), (2,2)],
    2: [(1, 1), (1,2), (2,1), (2,2)],
    3: [(2, 1), (2,2)]
}

```

Inicialização do Solver

Queremos construir um horário semanal para o plano de reuniões de projeto de uma “StartUp” de acordo com certas condições, já definidas acima. Para tal usaremos o SCIP.

Começamos por criar 3 variáveis que nos vão auxiliar no processo:

- $reuniao_{s,d,t,p}$ terá valor 1 sempre que uma reunião do projeto p estiver agendada na sala s , no dia d , no tempo t (valor 0 caso contrário)
- $colaborador_{c,d,t,p}$ terá valor 1 sempre que o colaborador c estiver a participar na reunião do projeto p , no dia d , no tempo t (valor 0 caso contrário)
- $haReuniao_{d,p}$ terá valor 1 sempre que houver uma reunião do projeto p no dia d (valor 0 caso contrário)

In [656...

```

from ortools.linear_solver import pywraplp
solver = pywraplp.Solver.CreateSolver('SCIP')

reuniao = {}
for s in range(1,S+1):
    for d in range(1,D+1):
        for t in range(1,T+1):
            for p in range(1,P+1):
                reuniao[s,d,t,p] = solver.BoolVar(f"reuniao[{s},{d},{t},{p}]")

```

```
colaborador = {}
for c in range(1,C+1):
    for d in range(1,D+1):
        for t in range(1,T+1):
            for p in range(1,P+1):
                colaborador[c,d,t,p] = solver.BoolVar(f"colaborador[{c},{d},{t},{p}]")

haReuniao = {}
for d in range(1,D+1):
    for p in range(1,P+1):
        haReuniao[d,p] = solver.BoolVar(f"haReuniao[{d},{p}]")
```

Adição das restrições do problema

Após a inicialização do solver e das variáveis auxiliares é necessária a adição das restrições para a construção do horário.

- Cada reunião ocupa uma sala durante um slot:

```
In [657... for s in range(1,S+1):
            for d in range(1,D+1):
                for t in range(1,T+1):
                    solver.Add(solver.Sum(reuniao[s,d,t,p] for p in range(1,P+1)) <= 1)
```

- Cada reunião tem associado um conjunto de participantes:

```
In [658... for c in range(1,C+1):
            for d in range(1,D+1):
                for t in range(1,T+1):
                    for p in range(1,P+1):
                        if c not in projetos[p][1]:
                            solver.Add(colaborador[c,d,t,p] == 0)
```

- Cada projeto realiza um dado número de reuniões semanais:

```
In [659... for projeto, input in projetos.items():
            lider, lista_colaboradores, n_reunioes = input
            solver.Add(solver.Sum(reuniao[s,d,t,projeto] for s in range(1,S+1) for d in range(1,D+1) for t in range(1,T+1)) == n_reunioes)
```

- O líder do projeto participa em todas as reuniões do projeto:

```
In [660... for d in range(1,D+1):
            for p in range(1,P+1):
                for t in range(1,T+1):
                    lider = projetos[p][0]
                    solver.Add(solver.Sum(reuniao[s,d,t,p] for s in range(1,S+1)) == colaborador[lider,d,t,p])
```

- Mínimo de 50% do total de colaboradores do projeto têm de participar da reunião:

```
In [661... for d in range(1,D+1):
            for p in range(1,P+1):
                for t in range(1,T+1):
                    lider = projetos[p][0]
                    lista_colaboradores = projetos[p][1]
                    solver.Add(solver.Sum(colaborador[c,d,t,p] for c in lista_colaboradores) >= 0.5 * len(lista_colaboradores) * colaborador[lider,d,t,p])
```

- Os colaboradores são colocados em slots conforme a sua disponibilidade:

```
In [662... for d in range(1,D+1):
            for t in range(1,T+1):
                for p in range(1,P+1):
                    for c in range(1,C+1):
                        if (t, d) not in colaboradores[c]:
                            solver.Add(colaborador[c,d,t,p] == 0)
```

- Cada colaborador só pode estar numa reunião em cada slot:

```
In [663... for c in range(1,C+1):
            for d in range(1,D+1):
                for t in range(1,T+1):
                    solver.Add(solver.Sum(colaborador[c,d,t,p] for p in range(1,P+1)) <= 1)
```

Definição do objetivo do problema

Decidimos maximizar o número de dias que cada projeto tem reuniões. Para tal é criada uma restrição na qual *haReuniao* terá valor 1 caso exista alguma reunião do projeto *p* agendada no dia *d*, e valor 0 caso contrário.

```
In [664... for d in range(1,D+1):
            for p in range(1,P+1):
                solver.Add(haReuniao[d,p] <= solver.Sum(reuniao[s,d,t,p] for s in range(1,S+1) for t in range(1,T+1)))
                for s2 in range(1,S+1):
                    for t2 in range(1,T+1):
                        solver.Add(haReuniao[d,p] >= reuniao[s2,d,t2,p])
```

```
In [665... solver.Maximize(solver.Sum(haReuniao[d,p] for d in range(1,D+1) for p in range(1,P+1)))
```

Procura da Solução através do solver

Por fim é invocado o solver com o método Solve que nos dirá se é possível ou não criar um horário com os inputs do problema.

```
In [666... status = solver.Solve()
if status == pywraplp.Solver.OPTIMAL:
    print("Foi encontrada uma solução para o problema.")
else:
    print("Não foi encontrada uma solução ótima para o problema.")
```

Foi encontrada uma solução para o problema.

Interpretação dos resultados

```
In [667... nReunioes = {p: 0 for p in range(1, P + 1)}

if status == pywraplp.Solver.OPTIMAL:
    for d in range(1, D+1):
        print(f"\n{'=' * 50}")
        print(f"Dia {d}")
        print(f"{'-' * 50}\n")

        for t in range(1, T+1):
            temReuniao = False
            print(f"Tempo {t}:")
            print(f"{'-' * 50}")

            for s in range(1, S+1):
                for p in range(1, P+1):
                    if (s, d, t, p) in reuniao and reuniao[s, d, t, p].solution_value() == 1:
                        participa = set()
                        for c in projetos[p][1]:
                            if colaborador[c, d, t, p].solution_value() == 1:
                                participa.add(c)
                        print(f"    Sala {s}:")
                        print(f"        • Projeto {p} com participação de {'', ' '.join(str(c) for c in participa)}")
                        nReunioes[p] += 1
                        temReuniao = True

            if not temReuniao:
                print(f"    Nenhuma reunião a decorrer" )

        print(f"{'-' * 50}\n")

    print(f"\n{'=' * 50}")
    print("\nNúmero de reuniões realizadas por projeto:")
    for p, quantas in nReunioes.items():
        print(f"    Projeto {p}: {quantas}")
```

Dia 1

=====

Tempo 1:

Sala 2:

- Projeto 3 com participação de 2, 3, 6

Tempo 2:

Sala 2:

- Projeto 2 com participação de 1, 2, 4

Tempo 3:

Sala 1:

- Projeto 3 com participação de 2, 3, 6

Tempo 4:

Sala 1:

- Projeto 2 com participação de 2, 4

Sala 2:

- Projeto 1 com participação de 8, 1, 3, 5

Tempo 5:

Sala 2:

- Projeto 1 com participação de 8, 1, 3, 5

=====

Dia 2

=====

Tempo 1:

Sala 1:

- Projeto 1 com participação de 1, 3, 5

Tempo 2:

Sala 1:

- Projeto 2 com participação de 1, 2, 4

Tempo 3:

Sala 2:

- Projeto 1 com participação de 1, 3, 5

Tempo 4:

Nenhuma reunião a decorrer

Tempo 5:

Sala 1:

- Projeto 2 com participação de 1, 2, 4

Sala 2:

- Projeto 3 com participação de 3, 6, 7

Dia 3

Tempo 1:

Sala 1:

- Projeto 2 com participação de 1, 2, 4

Tempo 2:

Sala 1:

- Projeto 1 com participação de 1, 3, 5

Tempo 3:

Sala 2:

- Projeto 2 com participação de 1, 2, 4

Tempo 4:

Sala 1:

- Projeto 3 com participação de 2, 3, 6

Tempo 5:

Sala 1:
• Projeto 1 com participação de 1, 5

Sala 2:
• Projeto 3 com participação de 2, 3, 6

=====

Dia 4

=====

Tempo 1:

```
Sala 1:
  • Projeto 3 com participação de 2, 3, 6
Sala 2:
  • Projeto 1 com participação de 1, 5
```

Tempo 2:

Sala 1:

- Projeto 2 com participação de 1, 2, 4

Sala 2:

- Projeto 2 com participação de 1, 2, 4

Sala 2:

- Projeto 2 com participação de 1, 2, 4

Sala 1:

- Projeto 3 com participação de 2, 3, 6

=====

Dia 5

=====

Sala 1:

- Projeto 3 com participação de 2, 3, 6

Sala 2:

- Projeto 3 com participação de 2, 3, 6

Sala 2:

- Projeto 2 com participação de 1, 2, 4

Sala 1:

- Projeto 3 com participação de 2, 3, 6

Sala 1:

- Projeto 2 com participação de 2, 4

Sala 2:

- Projeto 1 com participação de 1, 3, 5

Número de reuniões realizadas por projeto:

```
Projeto 1: 8
Projeto 2: 11
Projeto 3: 10
```